

w12L63 POMDP Introduction

So far we were making assumptions

- At every point in time agent knows exactly where he is in the state space
- State space is Markov

Did we violate it sometime?

- When we talked about value function - looks at in parameterised form - the agent knew where it was

Such assumption is often not valid in practice:

- We are talking about state space
- In hierarchy - passenger is in car or at rugby, i gave up information so I can have compact representation for navigation
- It was not a great loss - how to navigate to y, features were irrelevant
- Still in Global context - we knew where agent is

Robot Navigation problem:

- Sonars, bump sensor, will get sonar reading
- Information is not measured for lot of reading/sensing information

I could look around and know everything:

- In cubicle - robot will know where it is in cubicle
- But will not where in building it is
- Will not know until i start moving around

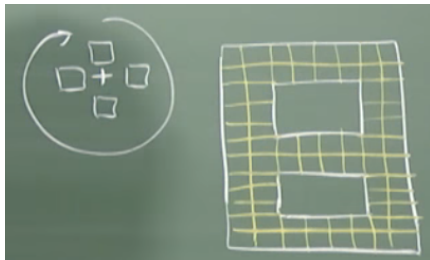
Humans / animal

- Perceptual aliasing - same perception from different view point
- Keep walking - here i saw - useful stuff like - men toilet - now i know where i am
- In big it campus get lost

Partial Observability:

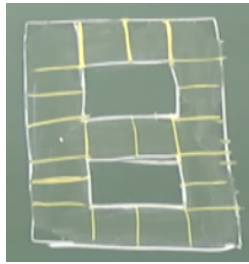
- Partial information is available about the state
- Not having full information about the state space

Agent case sense 4 nearby squares in clockwise direction"

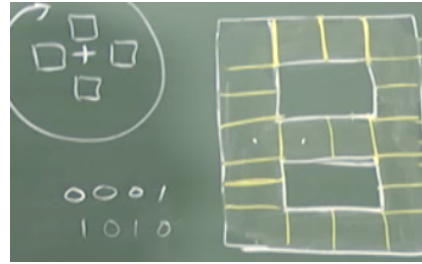


All are clear = 0 0 0 0 (0 = i can move in this direction)

1 0 0 0 or 0 0 1 0 (1 = there is no square, can't move)



0 0 0 1 (at dot square)



How many states do I have? 20 states? 1 0 1 0 - top and bottom, 0 1 0 1 - left and right
This is all the input i get

No. of time in 1st column	You have only 0 1 0 1 observation Can you tell me where you are? - Can be in any of 8 places I am going North and i see - 0 1 0 1 - One of 4 places	Go up again see 1001 Without history you can know As we have more history - Uncertainty comes down Sequence is uniquely identifiable Even individual observations may not be
----------------------------------	---	--

One way to tackling partial observability

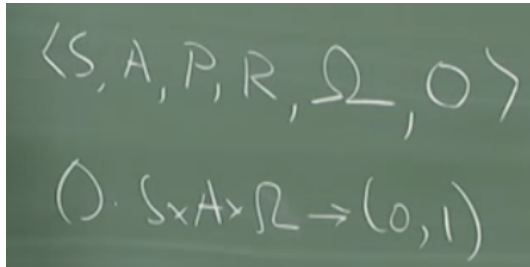
- Have some amount of history, it will help
- History allows you to figure out where you are

Probability was 1/8 Then it was 1/4 As get more observation, my belief In state space becomes focused - Assumption, i have mdp, some underlying state space (hence giving prob) - I can make observation based on some function of states - Based on that will form some belief about where i am in state space Then i can: solve, get policy,	
---	--

Different ways of solving Partially Observable problem

Standard model: POMDP (good model to know)

- MDP + some additional component for handling PO
- S, A, P, R
- Additionally will have omega - state of symbols ($(0, 1)^4$,
 - Or explicitly enumerate them as above smaller subside (half 8 out of 16)
- O - observation function - how states in omega are related
- mapping function: S, A, Omega to (0, 1)



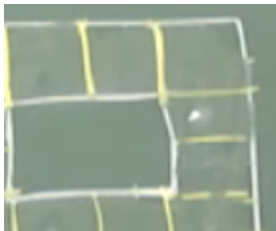
O could be probabilistic (could be fn of state & action)

We have assumed probability 0 or 1

23min: Sensor with noise of 0.1 - 10% of time it will report incorrectly (0->1, or 1->0)

Now my state space has gone to 16 (2^4)

- Anything can come at any time



- prob of 0101 is $0.9^4 / 8$
- That is the prob i will see 0001 which is $0.9^3 * 0.1$ (3 correct, one is incorrect)

Can have prob distribution over observations

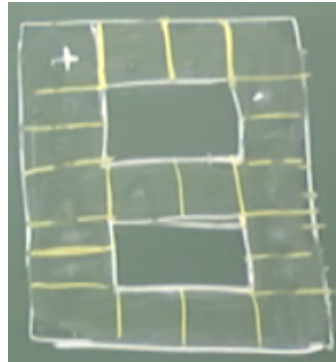
- Go back to simple exercise

What will be belief state:

- Will 0101 still have support of 8?
- They will still have bulk of probability
- I can get 0101 anywhere in the world - more and more sensors have to misfire
 - For 1010 -> all sensor have to misfire 0.1^4
 - If previous 3 misfired ($0.1^3, * 0.9$)
 - Assuming motion is deterministic - new observation could be noisy
 - Computation becomes more complicated

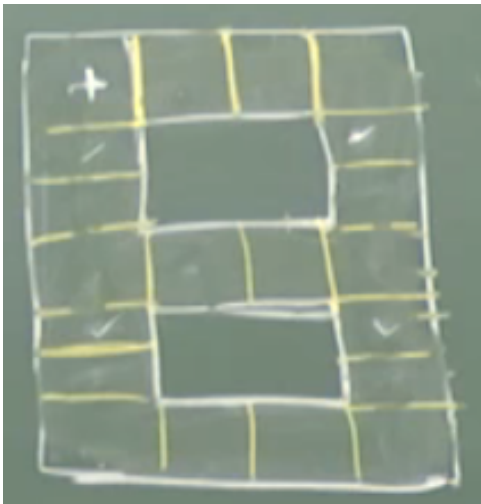
How do we do belief update - compactly?

From $1/8$ to will become $1/4$ -> it will become lower - will be discounted

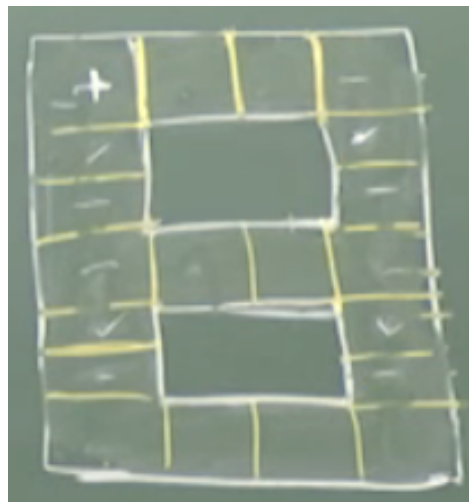


All 4 corners will have highest probability:

Prob where I am



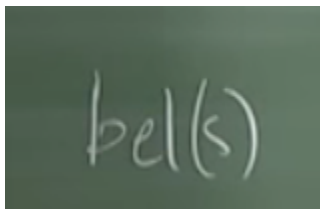
highest will be 2nd from top left and right and 3rd up from bottom left and right



2nd highest will with dashes

Everything else will have least prob
Horz-Cross corridor will be almost zero $0.1^4 * 0.1^4$

Belief State = prob that you are occupying state s :



w12L64 Solving POMDP

Different ways of solving POMDP

1. History based methods

- Does't use Belief state
- Use history directly
- Semi - Markov option - look at the history since the state started - so states are

last is recent state • Learn Q of	• Can have actions as well as part of history Going up twice will give more information - My sensors are stuck at 0101 - Two ups still get 0101 - noisy - Actions help disambiguate Keep doing up down - 0101 are fine Keep doing up-up - seeing 0101 - there is something wrong
--	--

Next time you start episode, go back to beginning

Disadvantage:

- Large state space (if episode is large)
- For parameterise representation - each state is of unequal length
- Wastage of computation - need to visit states often
- Can use LSTM to model sequence model not HMM

When do you think will it work?

- What we require of state in mdp?
- Markoveness definition:
 - Prob of s_{t+1} given s_t , a_t is $\rightarrow$?
 - Sufficient, enough will determine will next state, what could have done with entire history
 - We want history to have more information than current state

Broken Vision System:

- I remember history it is 0101, 0101 ..
- Don't get state information - significant level of noise introduced
- Is it Markov or not
 - NO, there is no underlying state space
 - I can predict next observation very clearly 0101 (blank screen)
 - Don't need history
 - It will not change, that's why it is Markov ?

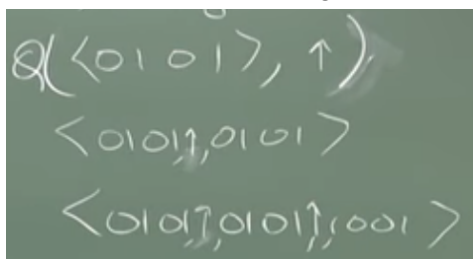
Setup k-th order Markov system:

- Depends on k previous states (k can be arbitrarily large)
- Then it will work

If NOT making any assumptions, then it is NOT written in problem!

Unequal state distribution:

- Use LSTM - will get fixed length history
- Or pad with No-op symbol (1, 1, 1, 1 - surrounded by obstacles)
- Or pick something which is not state (-1, -1, -1, -1)



Where is the wastage of computation?

- In noise free scenarios - have trajectory of 10 states, state 9 is 1001
 - I am able to localise perfectly
- Do i need to remember 1 to 8
 - Not needed
- Sometime i will need history of 2, 3, etc.
- If you fix k - you will compute beginning states un-necessarily

Prob with k-th order Markov

- Determining k
- Either have large
- Or too small - you will miss out the information

Adv:

- Easy to use - don't have to worry about complicated things
 - E.g. belief state is complex operation - how do you do update
- Intuitive to understand
- Lot easier to implement
- If mdp has small k - pretty good at solving problem

U-trees: (Utile Trees):

- Trees all states as one thing to begin with
- As you get more observations and rewards
- Make better predictions
 - I will worry about first bit - in one state it is 0, in other state 1
- Should i look at second bit or
- First bit few steps ago (history)
- Last step was 1001 - will not be able to make any distinction - i know outcome
- If outcome was 1010 - need to look one more step back
 - If it 1010 i am done
 - If it is 1010 again - i need to look at one more step back
- What length of history i will use will depend on
- It is wasteful of data - was giving statistical independence
 - Not using same trajectory
 - Give independence

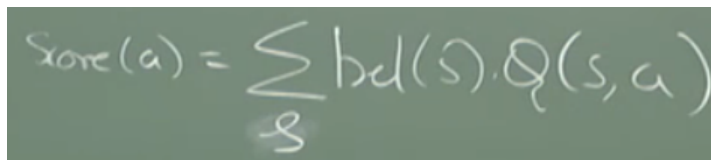
Amazingly Powerful:

- Solved Non-trivial - which PO will struggle with

2nd class of Approaches:

Q-MDP like approaches:

- Assume MDP is available to you
- Since you know mdp (SAPR is known), go ahead and solve it, get policy
 - Do value iteration, policy iteration
- I can solve the MDP but
 - **Execution becomes a problem (todo?)**
- **How do I execute?** Way of solving it - have POMDP
 - Solve mdp component
 - For execution what do i do
 - We found Q function (ran sarsa, value fn, q learning, policy iteration - have q* for mdp)
 - **Belief:** bel(s) - prob of being in state s
 - Some kind of score - you are in current belief state


$$\text{score}(a) = \sum_s \text{bel}(s) \cdot Q(s, a)$$

- Action will be:

$$\text{Score}(a) = \sum_s \text{bel}(s) \cdot Q(s, a)$$

$$\left\{ \begin{array}{l} \max_a Q(\text{bel}(s), a) \\ \rightarrow Q(\text{bel}(s), a) \end{array} \right.$$

-
- Or can think of it as above
- You are in current belief state - value of taking action a
- Now solved Q, converted to belief state, action

All other methods except history - will assume you know POMDP

- Bayesian will start with some distribution

Prob with this method:

- Maintain belief state

Only one policy

- You have mdp you know how to solve it
- Now you are doing heuristic based on state
 - **Policy may not be optimal - given uncertainty in state**

Dumb way of thinking:

- Have mdp - compute belief
- Can Define mdp - where states are belief states
- Transitions between belief states is given by computation for finding belief states
 - Given action what will be next state
 - Will become deterministic
- Mdp - i know previous belief state, observation, current action - will know what next state will be
- Problem with solving that mdp:
 - Have very high dimensional continuous state space
 - Originally have million, now million dimensional simplex, crazy state space
 - Need to learn value function

Next class - how to solve belief state mdp:

- Use regularity available in belief state mdp
- Very fact it is simplex
- Value fn have to be related
- Learning policy from history to action

Aware of whole history, aware of uncertainty - better

Adhoc things are easier to implement

- In practice not too bad
- In real world problems Q-MDPs give good enough solution

Prob:

- You can't prove anything

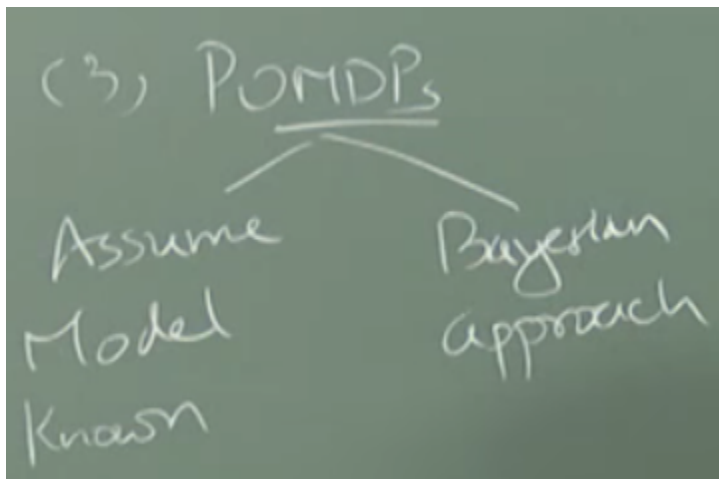
34min

3rd class of problem - directly solving mdp

1 assumes that model is known

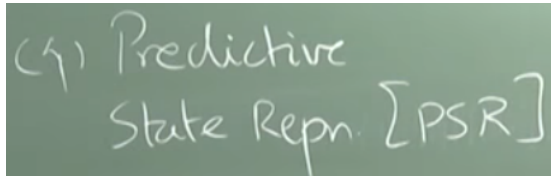
2 Bayesian approach - don't assume the model

- Assume some kind of distribution
- As you gather data, you narrow down
- You can generate some model, use that experience



4th class of algorithm -

Refer Video



Turns whole idea of history based method on its head

Why i am bothered about history

- Why do i need history to make future predictions

What i need to know about future

- If you make certain assumptions about the dynamics
- Go forward two steps - will get a wall
- Left 5 steps - get a wall
- Right step and then u[p - hit a wall
- if I know prob of all these then i know where i am
- After 3 steps - then in 1st and 2nd step will not hit a wall
- These are tests (observation and their prob sequences)
 - Action north, obs clear
 - Action west, obs wall

PSR consists of tests

- Don't assume MDP (as POMDP does) - will need some of the history
- Which is enough for me to represent world
- Tests are adequate to make any decision
- Very Powerful

If you build PSR on POMDM

- PSR can go well beyond PMDP - they don't need
- Get direct POMDP solution

They don't need state space

- Satinder Singh - propose

It is hard to learn PSR

- You can learn liner PSRs
- Set of test of tests, i can recover probability of being true
- Based on 3 test, using probabilities can give prob
- No. of linear tests = no. of state

In nonlinear relation

- Product or divide one by other
- Get a representation which is smaller than no. of states
- Finding non-linear psrs is hard

Look at Deep RL stuff

Next class:

